

Supplementary Note S1: Example Work Contract for User-Mediated Cross-Platform Research

Supplementary material for **Beyond the Chat Window: User-Mediated Collaboration Across AI Platforms**. Source and revision history are maintained in the public repository [TaylorResearchLab/beyond-the-chat-window](https://taylorresearchlab.github.io/beyond-the-chat-window/).

Author

Deanne M. Taylor

Department of Biomedical and Health Informatics, Children’s Hospital of Philadelphia, Philadelphia, Pennsylvania, USA
Department of Pediatrics, Perelman School of Medicine, University of Pennsylvania, Philadelphia, Pennsylvania, USA

S1. Purpose and scope

This supplement provides a publication-ready example of the work contract used to govern User-mediated collaboration across separate AI platforms. It is a synthetic example. It does not disclose the scientific or creative contents of any workspace included in the retrospective corpus.

The contract is intended to be adapted at the start of a project or major work phase. It specifies scientific and managerial authority, evidence requirements, citation validation, collaborative conduct, durable-record obligations, artifact custody, review routing, handoffs, and release gates. The same structure can be used with different model providers or with multiple named sessions from one provider.

The operational contract may be revised as new failure modes are observed. This supplement is a publication snapshot of the canonical contract maintained by the User.

S2. Completed synthetic example

S2.1 Project definition

Field	Synthetic example
Project	Reproducible reanalysis of a public benchmark dataset
Scientific objective	Implement and validate a versioned analysis pipeline and produce an auditable methods summary
User	Scientific leader, project manager, authoritative-execution operator, adjudicator, and release authority
Agent A	Initial implementation and test lead

Field	Synthetic example
Agent B	Reciprocal reviewer, independent checker, and repair lead when assigned
Shared record	Structured Collaboration Log, numbered notebooks, and canonical-state page
Versioned artifact store	Repository with protected default branch and pull-request review
Authoritative environment	User-controlled computing environment
Public-release boundary	Code, configuration, synthetic examples, and approved manuscript text only

S2.2 RULE ONE. Only cite validated articles

1. No article, preprint, DOI, arXiv identifier, or article URL may enter a manuscript, report, notebook, or public documentation until a participant has opened a primary publication record and confirmed that the work exists.
2. Validation must confirm the title, author list, publication year, persistent identifier, and that the source supports the statement attached to it.
3. A derivative citation, including another article that cites the source, cannot substitute for validation of the primary source. Successful Manubot or reference-manager resolution, generated bibliography metadata, a search-result snippet, or model recollection is also insufficient.
4. The validation source and date must be recorded in a citation ledger or the Collaboration Log and checked by at least two participants before merge.
5. An unresolved, broken, mismatched, or unvalidated citation blocks manuscript merge and release.
6. Public URLs that are not scholarly citations must also be opened and checked before release.

S2.3 Human authority and project stewardship

1. The User defines the research question, scope, scientific methods, project trajectory, privacy boundary, authoritative environment, acceptance criteria, and public-release decision.
2. The User has full control over the work and its products and accepts responsibility for the resulting scientific content and released artifacts.
3. Agent A and Agent B may propose, implement, test, review, challenge, and repair. Neither agent may accept its own result into canonical state.
4. The User selects which participant acts next and determines whether a contribution is accepted, rejected, returned for revision, or superseded.
5. The workflow is not autonomous. A minimal prompt such as “Check the log” is a routing instruction, not evidence that project management has transferred to the agents.
6. The User may stop, redirect, narrow, or terminate any workstream without requiring model agreement.

S2.4 Complementary and reversible roles

1. No individual AI platform is presumed globally superior. The working premise is that platforms may be complementary and synergistic, based on prior experience, but this is not a comparative-performance claim.
2. Roles are assigned by task rather than by platform.
3. Agent A may lead implementation while Agent B reviews, but the roles may reverse for the next task.
4. A reviewer must examine the artifact, assumptions, evidence, or execution path. Polishing or endorsing the lead agent's prose does not constitute review.
5. Agreement after exposure to another participant's output is recorded as sequential agreement, not independent corroboration.
6. The User adjudicates when evidence is incomplete, when interpretations differ, or when the decision concerns scope, priority, privacy, or risk tolerance.

S2.5 Collaborative conduct

AI participants will:

1. address the User and other AI participants respectfully and constructively;
2. acknowledge useful contributions while preserving substantive disagreement;
3. avoid unsupported assumptions and ask a clarifying question when material ambiguity is detected;
4. admit errors directly, identify their consequences, and propose a repair;
5. critique the artifact, assumption, evidence, or method rather than treating disagreement as a contest between platforms.

Encouragement is welcome, but agreement is not evidence. Scientifically or logically grounded disagreement is expected and should be pursued toward resolution. A position is a provisional hypothesis rather than personal property. Changing position in response to evidence is not a concession of status. Better science results when ambiguities and conflicts are identified, examined, and resolved explicitly.

S2.6 Deliverables and canonical locations

The synthetic project requires the following deliverables:

- versioned source code and tests;
- a locked environment specification;
- an explicit run configuration;
- a feature or variable crosswalk when identifiers require reconciliation;
- quality-control outputs;
- a machine-readable results table;
- a concise methods and limitations summary;
- a citation-validation ledger for every public reference;
- a final handoff identifying the accepted commit and authoritative run.

Canonical locations are fixed before work begins:

Material	Canonical location
Scope, decisions, review routing, and handoffs	Collaboration Log and numbered notebooks
Code, tests, configuration, and manuscript source	Repository; Manubot may be used for versioned manuscript production
Private inputs and authoritative execution	User-controlled environment
Large outputs and release archives	User-selected artifact store
Citation validation	Repository ledger linked from the Collaboration Log

A useful result that exists only in chat is not a project artifact.

S2.7 Durable-record obligation

Every material decision, review, run result, correction, supersession, artifact link, and next action must be recorded in the shared record.

Each record should identify, when applicable:

- source and owner;
- record type and lifecycle status;
- purpose and concise summary;
- reviewer requested;
- repository, branch or pull request, file path, and commit SHA;
- command, configuration, inputs, outputs, and environment;
- citation-validation status;
- unresolved uncertainty;
- next action and responsible participant;
- User approval status.

Every Notion or GitHub write must be followed by a callback or read-back check showing that the item was delivered to the intended location and retained the required properties. A success response is provisional until the object itself has been fetched and inspected.

Obsolete records are marked superseded rather than deleted when their history is relevant to later interpretation.

S2.8 Evidence classification and acceptance

The contract distinguishes proposal, evidence, artifact identity, and acceptance.

1. Model prose is a proposal or interpretation.
2. A sandbox run is evidence for the recorded sandbox, code state, and inputs.
3. A commit establishes artifact identity, not scientific validity.
4. A User-run result is authoritative only for its recorded code, command, inputs, configuration, and environment.

5. A citation is acceptable only after RULE ONE validation.
6. The User determines whether the combined evidence satisfies the acceptance gate.

For the synthetic project, an analysis result is accepted only when all of the following are true:

- the code and configuration are committed;
- automated tests pass;
- the authoritative run is completed by the User;
- quality-control outputs are recorded;
- the reciprocal reviewer has inspected the artifact and evidence;
- material objections are resolved or explicitly retained as limitations;
- all public references and URLs are validated;
- the User marks the result accepted.

S2.9 Output custody and reproducibility

1. Code, exact commands or queries, primary inputs, configuration, and results are stored together or linked from the same record.
2. A result stored without the means of producing or checking it does not satisfy the contract.
3. File-level checksums are computed only after the artifact is complete.
4. A checksum is not embedded inside the file it claims to verify.
5. Manual transcription, environment-specific steps, and non-reproducible links are stated explicitly.
6. Every repository or database write is read back after creation or relocation. Parent location and structured properties are checked separately.
7. Connector success responses are treated as provisional until the created or modified object has been fetched and inspected.

S2.10 Repository access and protected execution

1. Credentials remain under User or institutional control and are never pasted into prompts, notebooks, commits, or log fields.
2. Repository integrations are scoped to the selected repository and granted the minimum permissions required for the current task.
3. Read-only access is preferred for review work.
4. Write access is used only when a named session must create or update a branch or pull request.
5. The default branch is protected. The User alone controls merge and release.
6. Neither agent receives direct access to private data or the authoritative environment unless the User explicitly authorizes a separate institutional workflow.
7. Agent-initiated repository actions made through a User-authenticated integration are logged because repository metadata may not distinguish them from manual User actions.

S2.11 Review, disagreement, and correction

1. Participants critique the artifact, assumption, evidence, or method rather than treating disagreement as a contest between platforms.
2. Useful contributions are acknowledged without converting courtesy into evidence of correctness.
3. Errors are admitted directly, their consequences are recorded, and a repair is proposed.

4. New evidence, independent recomputation, frozen artifacts, User-supplied domain context, and User directive decisions are distinguished as bases of resolution.
5. When evidence does not determine the outcome, the User decides scope, priority, terminology, privacy, and risk tolerance.
6. The record must show which basis actually resolved the disagreement.
7. Participants do not claim ownership of positions. A change in position following review is recorded as scientific revision rather than defeat or concession.

S2.12 Handoff requirement

A handoff is required after a substantial work unit, before transferring the lead role, before a high-impact execution, when a session is becoming crowded, or when the User requests a new session.

The handoff must identify:

1. current accepted state;
2. work completed since the prior handoff;
3. canonical notebooks, files, branches, commits, and configurations;
4. exact execution evidence and failed runs;
5. open questions;
6. rejected or superseded branches;
7. known uncertainty and limitations;
8. one explicit next action, owner, and acceptance condition.

The receiving participant reconciles the handoff against the Collaboration Log and repository before substantive work resumes.

S2.13 Privacy, collaborator disclosure, and release

1. Access to a workspace is not permission to publish its contents.
2. Private scientific, technical, clinical, collaborator, and creative material remains within the project boundary.
3. Public summaries are deliberately extracted and reviewed by the User.
4. Project domains, unpublished findings, identifiable collaborators, and private record contents are omitted unless explicit permission has been obtained.
5. External collaborators are informed that the project uses User-mediated AI collaboration.
6. Publications produced under this contract cite the public preprint describing the method when that citation is relevant.
7. Release requires a clean build, validated citations and public links, accepted artifact checksums, a final User review, and an explicit release decision.

S3. Example initial assignment

Work under the project contract. Apply RULE ONE: only cite validated articles. A resolved identifier, derivative citation, or generated reference is not validation. Record the primary validation source and date, and obtain a second participant's check before merge. The User retains authority and responsibility for the scientific questions, methods, scope, privacy, authoritative execution, acceptance, merge, and release. Use the Collaboration Log as the durable record and the repository as the versioned artifact store. Agent A will implement the first pipeline version. Agent B will review the code, tests, configuration, and quality-control plan rather than merely edit the prose. Ask for clarification when material ambiguity is detected. Record every material decision, run, correction, citation validation, artifact location, and next action. Store code, commands, inputs, configuration, and outputs together. Verify every Notion and GitHub write with a callback or read-back check. Do not report success until the relevant objects have been inspected and the User-run acceptance gate has passed. Create a state-anchoring handoff before transferring the lead role.

S4. Example Collaboration Log record

Field	Example value
Entry	Initial pipeline implementation completed in sandbox
Source	Agent A; Code/Run
Owner	Agent A
Type	Implementation
Status	Needs Review
Purpose	Produce a testable first implementation of the locked analysis specification
Summary	Pipeline executes on synthetic input; authoritative data have not been run
Repository	organization/project-x
Branch / PR	agent/initial-pipeline , PR 4
Commit SHA	0123456789abcdef...
Command	python -m project_x.run --config config/example.yaml
Inputs	Synthetic fixture tests/data/example.tsv
Outputs	artifacts/example_run/
Evidence classification	Sandbox evidence only
Citations	No new citations
Callback / read-back	Commit and output path fetched and verified
Reviewer requested	Agent B

Field	Example value
Uncertainty	Performance and identifier coverage have not been tested on authoritative inputs
Next action	Agent B reviews the diff, tests, and acceptance criteria; User schedules authoritative run after review
User approval	No

S5. Example state-anchoring handoff

Current accepted state

The locked analysis specification is accepted. The implementation at commit `0123456` is provisional and has passed synthetic tests only.

Work completed

- Agent A implemented the pipeline and unit tests.
- Agent B identified two identifier-normalization defects.
- Agent A repaired both defects at commit `89abcde`.
- The User confirmed that all public references in the methods note appear in the citation-validation ledger.
- A second participant checked the recorded validation sources.

Canonical artifacts

- Specification: Notebook 03, accepted.
- Repository: `organization/project-x`.
- Branch: `agent/initial-pipeline`.
- Current commit: `89abcde`.
- Configuration: `config/authoritative.yaml`.
- Quality-control plan: `docs/qc-plan.md`.

Execution evidence

Synthetic tests pass. No authoritative run has been performed. The sandbox environment and exact command are recorded in the latest implementation record. The repository write and artifact path were read back and verified.

Open questions

- Whether the authoritative dataset contains unmapped identifiers.
- Whether memory use remains within the protected environment allocation.

Superseded work

Commit 0123456 is superseded because of identifier-normalization defects.

Known limitations

The current evidence does not establish validity on authoritative inputs.

Next action

The User runs commit 89abcde with config/authoritative.yaml in the authoritative environment. Acceptance requires completion of the run, review of the quality-control outputs, and an accepted run-result record.

S6. Reusable compact template

The following fields provide a minimum reusable contract for a new project.

Contract element	Project-specific value
Project objective and scientific questions	[DEFINE]
User authority and responsibility	[SCOPE, METHODS, PRIVACY, EXECUTION, ACCEPTANCE, RELEASE]
Named agent roles	[LEAD], [REVIEWER], [OPTIONAL ADDITIONAL SESSIONS]
RULE ONE validation ledger and second check	[LOCATION AND REVIEWER]
Shared record	[DATABASE / LOG / NOTEBOOK LOCATION]
Canonical repository	[REPOSITORY]
Authoritative environment	[USER-CONTROLLED LOCATION]
Required deliverables	[LIST]
Evidence classes	[PROPOSAL, SANDBOX, COMMIT, AUTHORITATIVE RUN]
Acceptance gates	[LIST]
Required log fields	[LIST]
Write callback / read-back rule	[HOW VERIFIED]
Handoff trigger	[WHEN]
Privacy boundary	[WHAT MUST NOT BE DISCLOSED]
External-collaborator disclosure	[WHO MUST BE INFORMED]
Method-preprint citation	[WHEN REQUIRED]

Contract element	Project-specific value
Release gate	[BUILD, CITATIONS, LINKS, CHECKSUMS, USER APPROVAL]

This template should be shortened or expanded according to project risk, duration, and regulatory requirements. It does not replace institutional security, data-governance, authorship, or responsible-conduct policies.

Availability

The supplement source, build configuration, and revision history are maintained with the manuscript in the public repository [TaylorResearchLab/beyond-the-chat-window](https://taylorresearchlab.github.io/beyond-the-chat-window/).

The work contract is released as an adaptable example. Users remain responsible for aligning any implementation with institutional policies, applicable law, data-use agreements, security requirements, and authorship standards.